

Project Design Phase-II

Data Flow Diagram & User Stories

Date	28 September 2025
Team ID	SWTID-2026-2056
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

In the context of Implementing Client Scripts and UI Policies on the Incident form in ServiceNow, DFDs illustrate how user interactions on the Incident form trigger client-side logic, how field values flow into conditions, and how the form state (visibility, mandatory, read-only) is updated in real time without server round-trips for pure UI Policy actions, or with limited server calls when Client Scripts use GlideAjax.

DFD Level 0 – Context Diagram (Incident Client-Side Logic)

Level 0 shows the overall system boundary for client-side behaviour on the Incident form.

External Entities:

- IT Service Agent / ITIL User – interacts with the Incident form (creates, updates, resolves incidents).
- End User / Caller – views limited Incident form (read-only or restricted fields via UI Policy).
- ServiceNow Platform (Server) – stores Incident records, executes ACLs, Business Rules, and responds to GlideAjax.

Process (System):

Incident Form Client-Side Engine (Client Scripts + UI Policies) – evaluates conditions on form load / field change, applies field attributes (mandatory, visible, read-only), shows messages, validates data, and optionally calls server via GlideAjax.

Data Stores:

- Incident Table (server-side) – persistent storage of Incident records.
- Client Form State (browser) – current field values, UI Policy actions, and temporary script variables.

DFD Level 1 – Detailed Processes

The Level 1 DFD expands the client-side engine into major processes that handle the implementation of Client Scripts and UI Policies on the Incident table.

Process	Description	Key Data Flows
1.0 Form Load	OnLoad Client Scripts and UI Policies (On load = true) execute. Initial field visibility, mandatory and read-only states are set based on current record values and user roles.	Incident record data → Client form state User role / view → UI Policy engine
2.0 Field Change	onChange Client Scripts fire when a field value changes. UI Policies re-evaluate conditions (e.g., State = Resolved). Actions: make Close notes mandatory, hide/show fields, set read-only, display alerts.	Old value / New value → Client Script Condition result → UI Policy Actions g_form API updates → Form UI
3.0 Form Submit	onSubmit Client Scripts validate data before save. Can prevent submission (return false) if rules fail (e.g., missing Close notes when State = Resolved).	Form field values → Validation logic Pass/Fail → Allow or block submit Valid data → Server (Incident table)

	UI Policies do not run on submit.	
4.0 Server Lookup	Client Scripts using GlideAjax or g_form.getReference() fetch additional data (e.g., caller details, CI info). Results update form fields or trigger further logic.	sys_id / query → Server Script Include JSON response → Client Script callback Updated values → Form fields

Note: Place actual diagram images (Level 0 context diagram and Level 1 process diagram) in this section following industry-standard DFD notation (Yourdon/DeMarco or Gane & Sarson). Typical flows include Agent → Incident Form → Client Script / UI Policy Engine → Form State / Server.

User Stories

Use the below template to list all the user stories for the product (Implement Client Script & UI Policy on the Incident form).

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance Criteria	Priority	Release
IT Service Agent	UI Policy – State Control	USN-1	As an agent, when I set the Incident State to Resolved, the Close notes field should become mandatory and the Opened by field should be hidden.	Close notes is mandatory; Opened by is hidden; Priority/Urgency become read-only when State = Resolved.	High	Sprint-1
		USN-2	As an agent, when I change the State away from Resolved, the previous field attributes (mandatory, visibility, read-only) should reverse automatically.	Reverse if false is enabled; fields return to original state when condition is no longer true.	High	Sprint-1
	Client Script – onChange	USN-3	As an agent, when I change the Priority field, the background colour of the Priority field should update to reflect severity (red for 1 – Critical, green for 5 – Planning).	Priority field background colour changes correctly based on the new value on the form.	Medium	Sprint-1
	Client Script – onSubmit	USN-4	As an agent, I should not be able to submit/resolve an Incident without filling Close notes when State is Resolved.	Form submission is blocked and an error message is shown if Close notes is empty while State = Resolved.	High	Sprint-1
	Client Script – Auto-populate	USN-5	As an agent, when I select a Caller, the Location and Contact number fields should auto-populate from the Caller's user record.	Location and phone fields are filled automatically after Caller is selected (via getReference or GlideAjax).	High	Sprint-1
Administrator	UI Policy Configuration	USN-6	As an administrator, I can create and activate a UI Policy on the Incident table with conditions, UI Policy Actions, and optional scripts (Execute if true / false).	UI Policy is active, applies on load and on condition change, and correctly controls field attributes on the Incident form.	High	Sprint-1
	Client Script Configuration	USN-7	As an administrator, I can create onLoad, onChange, onSubmit and onCellEdit Client Scripts for the Incident table and set UI Type (Desktop / Mobile / All).	Scripts execute only on the selected UI Type and table; order of execution is controllable.	High	Sprint-1
	Script vs Policy Decision	USN-8	As an administrator, I can choose UI Policy for simple	Documentation and implementation follow	Medium	Sprint-2

			declarative field control and Client Script when complex logic, old-value access, or onSubmit validation is required.	best-practice guidance (UI Policy preferred when no scripting is needed).		
End User / Caller	Restricted Form View	USN-9	As a caller viewing my own Incident, certain technical fields (Assignment group, Priority, Work notes) should be hidden or read-only based on UI Policies.	Only appropriate fields are visible/editable according to role and UI Policy conditions.	Medium	Sprint-2
Service Desk Manager	Major Incident Controls	USN-10	As a manager, when an Incident is marked as Major Incident, additional fields (Impact assessment, Communication plan) become visible and mandatory via UI Policy.	Major Incident related fields appear and are mandatory only when the Major Incident flag is true.	High	Sprint-2
Customer Care Executive						

Implementation Notes

- Prefer UI Policies for simple show/hide, mandatory and read-only behaviour (no-code, Reverse if false support).
- Use Client Scripts when you need access to the previous field value, complex conditional logic, onSubmit validation, or GlideAjax server calls.
- UI Policies execute after Client Scripts; conflicting logic is resolved in favour of the UI Policy.
- Always test on both Desktop and Service Portal / Mobile if UI Type is set to All.
- Client Scripts and UI Policies can be bypassed; critical security rules must also be enforced with ACLs and Data Policies.